

Background Research for Backend Design, Test Rule Derivation, and Tool Comparison

1. Introduction

In this project, we implemented an OpenAPI-based backend for API test generation and execution. Our backend accepts an OpenAPI 3.0 or swagger 2.0 file, parses it into an internal representation, generates a fixed set of test cases, and exposes the result through a FastAPI service. This document records the background research that informed those choices.

We use this document to record the research that shaped the backend design, the rule groups in the generator, and our comparison with other commonly used tools. Our design follows the OpenAPI model directly. The OpenAPI Specification defines a standard interface description for HTTP APIs and states that both humans and computers should be able to understand a service from the description alone [1]. This gave us the starting point for both the parser and the test generator. We also rely on JSON Schema style constraints such as type, enum, minimum, maximum, minLength, and maxLength when generating negative and boundary tests [2].

2. Current Backend Design

Our backend has two main parts.

1. The parser reads one or more api specs and extracts endpoints, parameters, request bodies, and response schemas.
2. The generator applies rule groups to produce test cases for each endpoint.

The parser is implemented in `src/spec_parser/parser.py`. It resolves local references, extracts path, query, and header parameters, and stores request and response schemas in an internal representation. The generator is implemented in `src/test_generator/generator.py`. It combines five rule sets:

- happy path
- negative
- boundary
- auth
- error status

This split is clear in the code. The generator calls `'generate_happy_path_cases'`, `'generate_negative_cases'`, `'generate_boundary_cases'`, `'generate_auth_cases'`, and `'generate_error_status_cases'` for each endpoint. We chose this structure because it keeps the rules explicit and makes the generated output easy to inspect.

Our backend service is implemented in `'src/backend/app/main.py'`. It provides routes for user registration, login, spec upload, spec history, and history deletion. This design supports a multi-user web application, which is important for our dashboard. The security model is also reasonable. OWASP lists broken object level authorization, broken authentication, and

improper inventory management among the key API risks [4]. Our user-scoped endpoints and history separation address these issues at a basic but useful level.

3. Background Research

The main source for our rules was the structure already present in the API description itself. We did not define categories first and then try to force the specification into them. Instead, we mapped common test needs to fields that are already defined by OpenAPI and JSON Schema.

“Happy path”:

Derived from valid required inputs and the example of the success response. This means the first happy path applies the rule sending the minimum valid request parameters generator by our rule based generator and if there is example response given, it will be used as input data for second happy path

“Negative missing”:

Derived from required query parameters and required body fields. If the contract says a field is required, then omitting it should be tested.

“Negative type”:

Derived from schema type. If a field is declared as integer, number, string, array, or boolean, then a wrong-type input is a useful failure case.

“Negative invalid”:

Derived from enum and format. If the contract restricts valid values or expected formats, then sending invalid values is a direct contract test.

“Boundary”:

Derived from explicit numeric, string, and array constraints such as minimum, maximum, minLength, maxLength, minItems, and maxItems.

“Auth”:

Derived from declared security requirements or authentication-style headers. If an endpoint requires authentication, then missing or invalid credentials should be tested.

“Error status”:

Derived from explicit error responses in the contract, especially 404. If the description declares a not-found case, then that behaviour should be tested.

This was the main line of reasoning behind the rule groups in our code. OpenAPI gave us the structure of each endpoint [1]. JSON Schema style validation rules gave us the basis for missing, invalid, and boundary inputs [2]. HTTP semantics gave us a standard basis for success and failure expectations [3].

4. Evidence From Our Current Output

Our generated files already show that the backend is producing structured output and not only theoretical rules. These counts come from `src/test_cases/user2/open_meteo_tests.json` and `src/test_cases/user1/petstore_tests.json`.

For the Open-Meteo sample, the generator produced 11 test cases:

- 1 happy path
- 2 negative type
- 2 negative missing
- 6 negative invalid

For the Petstore sample, the generator produced 92 test cases:

- 26 happy path
- 11 negative type
- 8 negative missing
- 22 negative invalid
- 15 boundary
- 3 auth
- 7 error status

These numbers matter because they show that our generator does more than basic schema validation. It creates several categories of tests from the same source contract.

For example, the generated output for PUT /pet includes:

```
{
  "title": "PUT /pet - Happy path: minimal valid request returns 200",
  "category": "happy_path"
}

{
  "title": "PUT /pet - Negative missing: required body field 'name' omitted",
  "category": "negative_missing",
  "expected_result": {
    "status_code_any_of": [400, 422]
  }
}

{
  "title": "PUT /pet - Boundary: body id max_int64(9223372036854775807)",
  "category": "boundary"
}
```

The generated output for GET /pet/{petId} includes a 404 case:

```
{
  "title": "GET /pet/{petId} - Error 404: non-existent petId",
  "category": "error_status",
  "expected_result": {
    "status_code": 404
  }
}
```

```
}  
}
```

This behaviour is consistent with HTTP semantics, where the meaning of methods and response classes is defined by the standard [3].

5. Code-Level Comparison With Other Tools

5.1 Schemathesis

Schemathesis is the closest tool to our backend in terms of input model. It reads an OpenAPI or GraphQL schema and generates tests from that schema [5], [6].

Typical Schemathesis usage looks like this:

```
import schemathesis  
  
schema =  
schemathesis.openapi.from_url("http://127.0.0.1:8080/openapi.json"  
)  
  
@schema.parametrize()  
def test_api(case):  
    case.call_and_validate()
```

This is a strong approach for runtime API testing. Schemathesis focuses on automatically exercising a live API and checking whether the implementation behaves correctly. It can detect issues such as server errors, invalid responses, schema mismatches, authentication problems, unsupported methods, and state-related failures.

However, although Schemathesis and our project both begin from an API specification, the two systems are designed with different goals and use different test generation logic.

The most important difference is the **generation strategy**. Our project uses a **rule-based approach**. Test cases are generated according to explicit categories such as happy path, negative type, negative missing, negative invalid, boundary, authentication, and error status. This means the generated suite is structured, predictable, and easy to understand. Each test case exists for a clear reason, and each one can be traced back to a specific contract rule or testing intention.

Schemathesis uses a **property-based and exploratory approach** instead. Rather than first producing a structured list of named test cases, it dynamically generates inputs during execution and sends them directly to the running API. Its purpose is not mainly to organise test cases for users, but to explore the behaviour of the implementation and discover failures automatically. In practice, this makes Schemathesis more execution-focused, while our project is more test-case-focused.

There are also important differences in how generated tests are presented. Our backend produces deterministic JSON test cases, and each case includes a stable test ID, title, category, input data, and expected result. This makes the output easy to display in the

frontend, review, edit, save, and trace back to the original contract. In contrast, Schemathesis mainly presents execution results, failure details, and reproduction commands. It can also export request and response traces, but it does not naturally provide all generated cases as a complete, structured, user-editable test suite in the same way as our system.

Another key difference is **visibility and control**. In our project, users can clearly see the generated test cases and modify them through the frontend. Schemathesis is mainly a CLI- and CI-oriented tool. It is strong for automated execution, but it is not designed as a front-end test case management system. Even when actual executed requests can be inspected through reports, they appear as execution traces rather than as a clean list of manageable test cases.

The two systems also differ in the way they treat **happy path** generation. In our project, happy path cases are explicitly generated in a structured form. These include minimal valid requests, example-based requests, and in some cases valid enum expansions, where several legitimate enum values are each turned into separate visible happy path cases. Schemathesis has an examples phase that is similar in spirit, because it can also use schema examples. However, it only uses examples that are explicitly available in the specification, unless extra configuration is applied. Therefore, its examples phase may cover only a small number of example requests, while our system can still generate a much broader set of happy path cases from rules even when the specification does not provide many examples.

A similar pattern appears in **boundary testing**. Our generator creates boundary cases when the specification contains explicit limits such as minimum, maximum, or size-related constraints. Schemathesis also has a coverage-oriented phase that can exercise constraints and boundaries. In this sense, some parts of our generator are similar to the examples and coverage phases in Schemathesis. However, the overall design is still different. Our generator organises test cases by testing intention and category, while Schemathesis organises execution by phases such as examples, coverage, fuzzing, and stateful testing.

Another important difference is that our project does **not** currently include fuzzing-style test generation. Schemathesis includes exploratory fuzzing and stateful testing, which allow it to generate many unusual inputs and multi-step sequences in order to discover runtime issues. This can reveal failures that are outside normal business scenarios, such as unsupported methods, malformed payload handling, or state-related bugs. These are useful for runtime robustness testing, but they are not the main focus of our project. Our system is more focused on systematic contract-based test generation and clear user-visible outputs.

The scope of the generated tests is also different. Our generator expands the specification into a broad visible set of test cases across many operations and categories. For example, it can systematically create cases for missing required fields, wrong parameter types, invalid enum values, boundary values, authentication failures, and expected error statuses. Schemathesis can also exercise many operations and input types, including path parameters, query parameters, headers, request bodies, unsupported methods, and stateful sequences, but these are generated dynamically during execution rather than being presented as a full visible suite from the start.

The workflow is also different. Schemathesis mainly targets a **running API implementation**. It requires a reachable API endpoint in order to actually execute generated requests. If only a specification file is available and no backend or mock server is running, Schemathesis cannot perform runtime testing. Our project is more contract-oriented. It can still generate structured test cases directly from the specification itself, even when no implementation is available. This makes our system useful earlier in the development process, when teams may want to inspect and prepare test cases before runtime integration is possible.

There is also a practical usability difference in **multi-spec support**. Schemathesis normally runs one specification at a time, and testing multiple specifications usually requires extra shell scripting or CI setup. Our project supports handling multiple specifications in a single workflow, which reduces manual effort and is more convenient for batch testing.

Overall, Schemathesis is strong for broad automated runtime exploration of a live API. It is effective at finding unexpected implementation failures and robustness issues through dynamic execution. Our project, however, is stronger in rule-based generation, deterministic outputs, structured test case visibility, frontend interaction, user control, multi-spec workflow support, and contract-focused traceability. Although both tools start from an API specification, Schemathesis is primarily an execution-focused runtime testing tool, while our project is a contract-oriented test generation and management system.

5.2 Postman and Newman

Newman is the command-line runner for Postman collections [7], [8]. It is widely used and easy to place in CI. A normal run looks like this:

```
newman run my-collection.json -e dev-environment.json
```

A failing status check is usually written as a scripted assertion:

```
pm.test("status is 400 when latitude is missing", function () {  
  pm.response.to.have.status(400);  
});
```

This is useful, but it highlights the difference between Newman and our generator. Newman executes collections well, but the collection itself still has to be written or exported. In other words, Newman is mainly an execution tool. Our backend does two jobs:

1. It derives test cases from the OpenAPI file.
2. It stores those cases in a consistent structure.

That makes our backend closer to a test generation system than a collection runner.

5.3 Dredd

Dredd is another description-based tool. Its documentation describes it as a command-line tool for validating an API description against a backend implementation [9]. Its model is simple:

```
dredd openapi.yaml http://127.0.0.1:8000
```

Dredd also supports hooks for setup and authentication:

```
const hooks = require("hooks");

hooks.beforeEach((transaction, done) => {
  transaction.request.headers["Authorization"] = "Bearer <token>";
  done();
});
```

Dredd is a useful comparison because it also starts from the API description. However, our own test showed that it was a weaker fit for the specifications we are using.

We tested Dredd directly against the same example specifications we already use in our project. We ran these checks locally with Dredd v14.1.0:

```
npx dredd src/specs/open_meteo.yaml https://api.open-meteo.com
npx dredd src/specs/petstore.json https://petstore3.swagger.io
```

In both cases, Dredd failed before it reached normal contract validation. For the Open-Meteo specification, it reported unsupported or invalid OpenAPI keys and then stopped because the required URI parameters latitude and longitude did not have example or default values.

```
error: API description URI parameters validation error in
C:\FYP\2026-csc1097-wangc9-siakj2\src\specs\open_meteo.yaml
(Open-Meteo APIs > /v1/forecast > 7 day weather forecast for
coordinates > 200 > application/json): Required URI parameter
'latitude' has no example or default value.
```

For the Petstore specification, it reported many unsupported OpenAPI keys and then stopped because required URI parameters such as petId, orderId, and username did not have example or default values.

```
error: API description URI parameters validation error in
C:\FYP\2026-csc1097-wangc9-siakj2\src\specs\petstore.json (Swagger
Petstore - OpenAPI 3.0 > /pet/{petId} > Find pet by ID. > 200 >
application/json): Required URI parameter 'petId' has no example
or default value.
```

This matters because our own backend parsed both specifications and generated test cases from them, while Dredd failed during description processing. The documentation still describes OpenAPI 3 support as experimental [9], and our local test results matched that limitation. The GitHub repository was also archived on November 8, 2024 and is now read-only [10].

5.4 Specmatic

We also reviewed Specmatic as a further comparison point because it is a current OpenAPI contract testing tool and is closer to our backend than a manual collection runner. The official documentation states that it can run contract tests from OpenAPI and can generate tests from the contract when examples are not available [11], [12]. Using Specmatic v2.42.0. We then ran it against our own sample specifications. For 'open_meteo.yaml', it parsed the contract and generated live requests, but some of those requests failed because values that matched the declared schema were still rejected by the real API. For example, one generated request produced the following result:

```
GET /v1/forecast?...&timezone=JOOXU&past_days=1
400 Bad Request
{
  "error": true,
  "reason": "Invalid timezone"
}
```

Specification expected status 200 but response contained status 400

This showed a different tradeoff from our backend. Specmatic explored the contract more broadly at runtime, while our backend generated a fixed and named set of cases up front. For example, our own output stored cases such as:

```
{
  "test_id": "TC-GET-v1-forecast-001",
  "title": "GET /v1/forecast - Happy path: minimal valid request
returns 200",
  "category": "happy_path",
  "steps": [
    {
      "action": "Send GET request to /v1/forecast with query
params {'latitude': 52.52, 'longitude': 52.52}"
    }
  ],
  "expected_result": {
    "status_code": 200
  }
}
```

This made Specmatic a stronger comparison point than Dredd in our tests because it handled both OpenAPI files at the contract level, but it also showed why our own backend was a better fit for a user-facing system where test cases need stable IDs, predictable inputs, and stored history [13].

6. How This Background Research Shaped Our Implementation

The main outcome of this background research was that we did not want a backend that only executed manual collections, and we did not want to depend on a tool that struggled with the same OpenAPI 3 specifications we were already using.

OpenAPI gives us a machine-readable API contract [1]. JSON Schema style constraints give us a way to create invalid, missing, and boundary cases [2]. HTTP semantics give us a standard basis for success and error expectations [3]. OWASP API guidance supports our decision to keep endpoints protected and user data separated [4].

Compared with Schemathesis, our backend searches less widely, but its output is more stable. Compared with Newman, our backend does more automatic generation from the API description. Compared with Dredd, our backend handled our example specifications more cleanly and did not stop at description-processing errors.

This background research also led us toward a rule-based structure:

- each category is explicit in code
- each generated case is visible in JSON output
- each endpoint can be traced back to the original OpenAPI input
- the backend API supports account-level history and repeatable uploads

This also matched the way we wanted the system to behave in the project. We wanted to upload a specification, generate a visible set of cases, store the result per user, and be able to trace each case back to the source contract. Our Docker and PostgreSQL setup also supports that workflow on another machine.

7. Conclusion

From this background research, we settled on a standards-based and deterministic backend test generator. It uses OpenAPI as the source description, applies clear rule groups, and produces structured test cases that the frontend can present to the user. For our project, this was a better fit than a pure collection runner, and for our current scope it was also a better fit than a search-heavy or archived alternative.

References

- [1] OpenAPI Initiative, "OpenAPI Specification v3.2.0," Sep. 19, 2025. [Online]. Available: <https://spec.openapis.org/oas/v3.2.0.html>. [Accessed: Mar. 5, 2026].
- [2] B. Hutton, H. Andrews, G. Dennis, and A. Wright, "JSON Schema Validation: A Vocabulary for Structural Validation of JSON," Jun. 16, 2022. [Online]. Available: <https://json-schema.org/draft/2020-12/json-schema-validation>. [Accessed: Mar. 5, 2026].
- [3] R. Fielding, M. Nottingham, and J. Reschke, "RFC 9110: HTTP Semantics," Internet Engineering Task Force, Jun. 2022. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc9110>. [Accessed: Mar. 5, 2026].
- [4] OWASP Foundation, "OWASP API Security Top 10 - 2023," 2023. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>. [Accessed: Mar. 5, 2026].

- [5] Schemathesis, "Pytest Integration," [Online]. Available: <https://schemathesis.github.io/schemathesis/explanations/pytest/>. [Accessed: Mar. 5, 2026].
- [6] Schemathesis, "schemathesis," GitHub repository. [Online]. Available: <https://github.com/schemathesis/schemathesis>. [Accessed: Mar. 5, 2026].
- [7] Postman, "Install and run Newman," [Online]. Available: <https://learning.postman.com/docs/collections/using-newman-cli/installing-running-newman/>. [Accessed: Mar. 6, 2026].
- [8] PostmanLabs, "newman," GitHub repository. [Online]. Available: <https://github.com/postmanlabs/newman>. [Accessed: Mar. 6, 2026].
- [9] Dredd, "Dredd latest documentation," [Online]. Available: <https://dredd.org/>. [Accessed: Mar. 6, 2026].
- [10] Apiary, "apiaryio/dredd," GitHub repository, archived on Nov. 8, 2024. [Online]. Available: <https://github.com/apiaryio/dredd>. [Accessed: Mar. 6, 2026].
- [11] Specmatic, "Contract Testing," [Online]. Available: https://docs.specmatic.io/documentation/contract_tests.html. [Accessed: Mar. 7, 2026].
- [12] Specmatic, "Contract Testing with Auth," [Online]. Available: https://docs.specmatic.io/documentation/contract_tests.html#contract-testing-with-auth. [Accessed: Mar. 7, 2026].
- [13] Specmatic, "specmatic," GitHub repository. [Online]. Available: <https://github.com/specmatic/specmatic>. [Accessed: Mar. 7, 2026].